# Blockchain Security Audit Report

# Table Of Contents

# 1 Executive Summary

On 2026.07.30, the SlowMist security team received the TuringBitChain team's security audit application for TBCNODE, developed the audit plan according to the agreement of both parties and the characteristics of the project, and finally issued the security audit report.

The SlowMist security team adopts the strategy of "white box" to conduct a complete security test on the project in the way closest to the real attack.

The test method information:

| Test method | Description |
| --- | --- |
| Black box testing | Conduct security tests from an attacker's perspective externally. |
| Grey box testing | Conduct security testing on code modules through the scripting tool, observing the internal running status, mining weaknesses. |
| White box testing | Based on the open source code, non-open source code, to detect whether there are vulnerabilities in programs such as nodes, SDK, etc. |

The vulnerability severity level information:

| Level | Description |
| --- | --- |
| Critical | Critical severity vulnerabilities will have a significant impact on the security of the DeFi project, and it is strongly recommended to fix the critical vulnerabilities. |
| High | High severity vulnerabilities will affect the normal operation of the DeFi project. It is strongly recommended to fix high-risk vulnerabilities. |
| Medium | Medium severity vulnerability will affect the operation of the DeFi project. It is recommended to fix medium-risk vulnerabilities. |
| Low | Low severity vulnerabilities may affect the operation of the DeFi project in certain scenarios. It is suggested that the project party should evaluate and consider whether these vulnerabilities need to be fixed. |
| Information | Features that are consistent with the design intent but important for users to be aware of. |

| Level | Description |
| --- | --- |
| Suggestion | There are better practices for coding or architecture. |

In black box testing and gray box testing, we use methods such as fuzz testing and script testing to test the robustness of the interface or the stability of the components by feeding random data or constructing data with a specific structure, and to mine some boundaries Abnormal performance of the system under conditions such as bugs or abnormal performance. In white box testing, we use methods such as code review, combined with the relevant experience accumulated by the security team on known blockchain security vulnerabilities, to analyze the object definition and logic implementation of the code to ensure that the code has the key components of the key logic. Realize no known vulnerabilities; at the same time, enter the vulnerability mining mode for new scenarios and new technologies, and find possible 0day errors.

# 2 Audit Methodology

The security audit process of SlowMist security team for smart contract includes two steps:

Smart contract codes are scanned/tested for commonly known and more specific vulnerabilities using automated analysis tools.

Manual audit of the codes for security issues. The contracts are manually analyzed to look for any potential problems.

Following is the list of commonly known vulnerabilities that was considered during the audit of the smart contract:

| NO. | Audit Items | Result |
| --- | --- | --- |
| 1 | SAST | Some Risks |
| 2 | Business logic security audit | Some Risks |

# 3 Project Overview

## 3.1 Project Introduction

TuringBitChain (TBC) is a Bitcoin-derived blockchain that uses the UTXO model, Bitcoin Script, and SHA-256 Proof-of-Work consensus. TBCNODE is its full-node software, providing peer-to-peer networking, transaction and block validation, mempool management, mining and block assembly, wallet functionality, and RPC services.

## 3.2 Coverage

Target Code and Revision:

https://github.com/TuringBitChain/TBCNODE/tree/v3.3.1

Initial audit version: afdbfc6ae8ad1f9e7d73d2d2cab0f3c98bc43fd3

Final audit version: c7e9d1c77bb0891a674ca3982f846e326947f5ad

Audit Scope:

```
TBCNODE/src/
├── chainparams.cpp
├── config.h
├── config.cpp
├── init.h
├── init.cpp
├── key.h
├── key.cpp
├── pubkey.h
├── pubkey.cpp
├── txmempool.h
├── txmempool.cpp
├── txn_validation_config.h
├── txn_validator.h
├── txn_validator.cpp
├── validation.h
├── validation.cpp
├── consensus/
```

```
│   └── params.h
├── mining/
│   ├── assembler.h
│   ├── assembler.cpp
│   ├── journal_change_set.h
│   ├── journal_change_set.cpp
│   ├── journal_entry.h
│   ├── journaling_block_assembler.h
│   └── journaling_block_assembler.cpp
├── net/
│   ├── net_processing.h
│   ├── net_processing.cpp
│   ├── validation_scheduler.h
│   └── validation_scheduler.cpp
├── policy/
│   ├── policy.h
│   └── policy.cpp
├── rpc/
│   ├── blockchain.cpp
│   ├── misc.cpp
│   └── net.cpp
├── script/
│   ├── interpreter.h
│   ├── interpreter.cpp
│   ├── sigcache.h
│   └── sigcache.cpp
└── wallet/
    ├── wallet.h
    └── wallet.cpp
```

## 3.3 Vulnerability Information

The following is the status of the vulnerabilities found in this audit:

| NO | Title | Category | Level | Status |
| --- | --- | --- | --- | --- |
| N1 | Nested `scriptPubKey` Serialization Does Not Inherit Parent Parameters | SAST | Suggestion | Fixed |

| NO | Title | Category | Level | Status |
| --- | --- | --- | --- | --- |
| N2 | `OP_CHECKDATASIG` Old and New Semantics Lack Consensus Activation Gating | Business logic security audit | Information | Acknowledged |
| N3 | Manager Key Rotation Lacks Height-Based Versioning | Business logic security audit | Information | Acknowledged |
| N4 | Dynamic Block Capacity Relies on the Collective Integrity of KYC-Approved Miners | Business logic security audit | Information | Acknowledged |
| N5 | Post-Genesis Script Validation Lacks an Independent Cost Limit | Business logic security audit | Low | Acknowledged |
| N6 | Whitelisted and Addnode Peers Can Bypass Misbehavior Disconnection | Business logic security audit | Low | Fixed |
| N7 | SHA-256 PoW Consensus Is Exposed to External Hashrate Migration and 51% Attacks | Business logic security audit | Low | Acknowledged |
| N8 | Extreme Fee-Rate Configuration May Cause a Non-Monotonic Fee Curve and Integer Overflow | Business logic security audit | Low | Fixed |
| N9 | Stale `ancestorsHeight` After Reorganization Invalidates Unconfirmed Transaction Chain Limits | Business logic security audit | Low | Fixed |

| NO | Title | Category | Level | Status |
| --- | --- | --- | --- | --- |
| N10 | Fixed-Offset Miner Bill Payment Address Parsing Allows Address-Binding Bypass | Business logic security audit | High | Fixed |
| N11 | Test-Only Block Validation Pause RPC Exposed in Production | Business logic security audit | Information | Fixed |

# 4 Findings

## 4.1 Vulnerability Summary

### [N1] [Suggestion] Nested `scriptPubKey` Serialization Does Not Inherit Parent Parameters

**Category: SAST**

**Content**

When calculating the output hash, `TxSerializeHash()` calls `SerializeSingleHash_OpNoCSize()` to serialize each `scriptPubKey`, but it always uses `SER_GETHASH` and version `0` instead of forwarding the caller-provided `nType` and `nVersion`. This causes the parent and nested serialization operations to use inconsistent type and version parameters. The current `CScript` implementation serializes only its bytes and does not depend on these parameters, so transaction hashes are not affected at present. However, if script serialization becomes dependent on the stream type or version in the future, this implementation may produce output hashes that differ from the caller's expectations, creating transaction identifier compatibility or consensus consistency risks.

- src/hash.h#L292-L298

```
CHashWriter ss_out(nType, nVersion);
for (const CTxOut &iout : obj.vout) {
    ss_out << iout.nValue;
```

```
    //ss_out << SerializeHash( iout.scriptPubKey, SER_GETHASH, 0);
    ss_out << SerializeSingleHash_OpNoCSize( iout.scriptPubKey, SER_GETHASH, 0);
}
```

**Solution**

Pass the parent's `nType` and `nVersion` to the nested serialization of `scriptPubKey`.

**Status**

Fixed

### [N2] [Information] `OP_CHECKDATASIG` Old and New Semantics Lack Consensus Activation Gating

**Category: Business logic security audit**

**Content**

The new implementation expands the single-byte `OP_CHECKDATASIG` flag to a two-byte format and uses it to select the message hash and the ECDSA or Schnorr signature algorithm, but the change is not activated through a block height or dedicated script flag. Because the opcode result determines whether a UTXO spend is valid, old and new nodes may produce different validation results if old-format transactions or unspent outputs exist on-chain, potentially causing historical revalidation failures, making old UTXOs unspendable, or creating a consensus split. If the old format has never appeared on-chain and all consensus nodes upgrade together, the issue is primarily a deployment compatibility risk.

- src/script/interpreter.cpp#L1744-L1762

```
LimitedVector &vchSig = stack.stacktop(-4);
LimitedVector &vchMessage = stack.stacktop(-3);
LimitedVector &vchPubKey = stack.stacktop(-2);
LimitedVector &vchFlag = stack.stacktop(-1);

if (!CheckDataFlag(vchFlag.GetElement(), flags, serror)) {
    // serror is set
    return false;
}
```

```
if (SignatureMethod signatureMethod = static_cast<SignatureMethod>(vchFlag[1]);
    !CheckDataSignatureEncoding(signatureMethod, vchSig.GetElement(), flags, serror)
||
    !CheckPubKeyEncoding(signatureMethod, vchPubKey.GetElement(), flags, serror)) {
    // serror is set
    return false;
}

bool fSuccess = checker.CheckDataSig(vchSig.GetElement(), vchMessage.GetElement(),
                                     vchPubKey.GetElement(), vchFlag.GetElement());
```

**Solution**

Activate the two-byte V2 format at a fixed block height and through a dedicated script verification flag. Continue executing the single-byte format according to its actual legacy semantics to preserve compatibility with historical transactions and existing UTXOs. Add tests covering behavior before and after activation, historical revalidation, and spending old-format UTXOs.

**Status**

Acknowledged; The project team stated that the single-byte `OP_CHECKDATASIG` flag was never used on mainnet. Pre-upgrade checks and post-upgrade monitoring found no compatibility issues or chain splits; therefore, this risk had no actual impact on mainnet.

### [N3] [Information] Manager Key Rotation Lacks Height-Based Versioning

**Category: Business logic security audit**

**Content**

KYC v2 validates all historical blocks against the Manager public-key set hardcoded in the current code without selecting key versions by block height. If old public keys are directly replaced or removed during rotation, new nodes and nodes running `-reindex` or `-reindex-chainstate` will be unable to validate historical blocks authorized by those keys and may fail to synchronize or rebuild because of `bad-miner-bill-v2`. Already synchronized nodes may continue operating, causing inconsistent validation results across node states or software versions. An

incompatible key rotation may prevent new nodes from completing IBD, prevent existing nodes from reindexing, and reduce node availability or split network validation results.

- src/validation.cpp#L847-L858

```
// Verify if fit one of the manager public key.
bool ret = false;
for (auto pubkeyManager : pubkeyManagerArr) {
    if (pubkeyManager.VerifySchnorr(msgHashManager, sigManagerVec)) {
        ret = true;
        break;
    }
}
if (!ret) {
    LogPrintf("Manager signature verification failed !!! \n");
    return false;
}
```

**Solution**

Maintain a block-height-based Manager key version table, define an explicit activation height for each rotation, and permanently retain old public keys for validating their historical validity ranges.

**Status**

Acknowledged; The project team stated that the Manager keys have not been rotated and all existing public keys remain valid; therefore, this risk has not been triggered. Height-based key versioning will be introduced for future key rotations.

### [N4] [Information] Dynamic Block Capacity Relies on the Collective Integrity of KYC-Approved Miners

**Category: Business logic security audit**

**Content**

`blockmaxsize` and `excessiveblocksize` allow miners and nodes to configure block generation and acceptance limits according to their hardware, network, and validation capabilities. The stability of this mechanism depends on

most KYC-approved miners reasonably configuring and publishing their capacity parameters and producing blocks as expected by the network.

A low-hashrate miner that generates a block exceeding most nodes' acceptance limits will generally have its block rejected, bear the loss of the block reward, and be unable to cause sustained network disruption. However, if an authorized miner controlling a significant or majority share of the network's hashrate continuously produces blocks that other nodes cannot accept, some nodes may stop synchronizing, the chain may fork, or network availability may decline. In this case, the primary risk arises from malicious authorized miners or hashrate concentration.

- src/init.cpp#L1830-L1837, L1844-L1851

```
// Configure excessive block size.
if(gArgs.IsArgSet("-excessiveblocksize")) {
    const uint64_t nProposedExcessiveBlockSize =
        gArgs.GetArgAsBytes("-excessiveblocksize", 0);
    if (std::string err; !config.SetMaxBlockSize(nProposedExcessiveBlockSize, &err))
{
        return InitError(err);
    }
}

// Configure max generated block size.
if(gArgs.IsArgSet("-blockmaxsize")) {
    const uint64_t nProposedMaxGeneratedBlockSize =
        gArgs.GetArgAsBytes("-blockmaxsize", 0 /* not used*/);
    if (std::string err;
!config.SetMaxGeneratedBlockSize(nProposedMaxGeneratedBlockSize, &err)) {
        return InitError(err);
    }
}
```

**Solution**

Continuously monitor the capacity parameters published by KYC-approved miners and their actual block-production behavior. Establish abnormal-block alerts, rapid authorization revocation, and incident-response coordination mechanisms. Gradually introduce independent miners to reduce authorized hashrate concentration and the potential

influence of any single miner.

**Status**

Acknowledged

### [N5] [Low] Post-Genesis Script Validation Lacks an Independent Cost Limit

**Category: Business logic security audit**

**Content**

`ConnectBlock()` counts and limits transaction and block sigops only before Genesis activation; the branch is skipped entirely afterward. At the same time, the consensus layer raises the limits on operations per script, script length, and multisig public-key count to `UINT32_MAX`. A miner can construct a high-complexity block that satisfies PoW while imposing unbounded script validation work on validators. An attacker who successfully mines such a block can force every validating node to perform extremely expensive script execution, causing prolonged CPU consumption, chain-tip stalls, and reduced network availability.

- src/validation.cpp#L4217-L4233

```
// After Genesis we don't count sigops when connecting blocks
if (!isGenesisEnabled){
    // GetTransactionSigOpCount counts 2 types of sigops:
    // * legacy (always)
    // * p2sh (when P2SH enabled)
    bool sigOpCountError;
    uint64_t txSigOpsCount = GetTransactionSigOpCount(config, tx, view, flags &
SCRIPT_VERIFY_P2SH, false, sigOpCountError);
    if (sigOpCountError || txSigOpsCount > maxTxSigOpsCountConsensusBeforeGenesis) {
        return state.DoS(100, false, REJECT_INVALID, "bad-txn-sigops");
    }

    nSigOpsCount += txSigOpsCount;
    if (nSigOpsCount > nMaxSigOpsCountConsensusBeforeGenesis) {
        return state.DoS(100, error("ConnectBlock(): too many sigops"),
            REJECT_INVALID, "bad-blk-sigops");
    }
}
```

**Solution**

Retain executable validation-cost limits for each transaction and block after Genesis. Use a combined accounting model for sigops, script bytes, and actual interpreter operations, and reject blocks that exceed the limits before entering expensive validation.

**Status**

Acknowledged

### [N6] [Low] Whitelisted and Addnode Peers Can Bypass Misbehavior Disconnection

**Category: Business logic security audit**

**Content**

When a peer's misbehavior score reaches the ban threshold, an ordinary peer is disconnected and its IP address is banned. However, whitelisted peers and peers configured through `-addnode` only trigger a warning log; the current connection is neither disconnected nor banned. The `fShouldBan` flag is also cleared, so subsequent misbehavior may not trigger the punishment flow again. Therefore, if the whitelist is overly broad, an Addnode peer is malicious, or a trusted peer is compromised, an attacker can continue sending invalid messages and increase the target node's CPU, memory, and network resource consumption.

- src/net/net_processing.cpp#L3550-L3568

```
if (state->fShouldBan) {
    state->fShouldBan = false;
    const CAddress& peerAddr { pnode->GetAssociation().GetPeerAddr() };
    if (pnode->fWhitelisted) {
        LogPrintf("Warning: not punishing whitelisted peer %s!\n",
                  peerAddr.ToString());
    } else if (pnode->fAddnode) {
        LogPrintf("Warning: not punishing addnoded peer %s!\n",
                  peerAddr.ToString());
    } else {
        pnode->fDisconnect = true;
        if (peerAddr.IsLocal()) {
            LogPrintf("Warning: not banning local peer %s!\n",
                      peerAddr.ToString());
```

```
        } else {
            connman.Ban(peerAddr, BanReasonNodeMisbehaving);
        }
    }
    return true;
}
```

**Solution**

Set `pnode->fDisconnect = true` to terminate the current connection when a whitelisted or Addnode peer reaches the misbehavior threshold. The special handling that avoids banning its IP address may remain, preventing a trusted peer from being permanently or repeatedly blocked from reconnecting.

**Status**

Fixed

### [N7] [Low] SHA-256 PoW Consensus Is Exposed to External Hashrate Migration and 51% Attacks

**Category: Business logic security audit**

**Content**

The chain uses Bitcoin-compatible double SHA-256 PoW and selects the best chain by cumulative work. Miner KYC verifies only whether a block producer is authorized and cannot constrain the behavior of an authorized miner. If a large KYC-approved SHA-256 miner redirects hashrate from networks such as BTC to this chain and controls a majority relative to the chain's honest hashrate, it can build a fork with greater cumulative work, causing chain reorganizations, double spending, transaction censorship, or reduced network availability. An external miner without valid KYC authorization cannot directly produce valid blocks, so the attacker must already be authorized or possess valid miner credentials. Actual exploitability also depends on the chain's current hashrate and the gap between it and externally transferable hashrate.

- src/pow.cpp#L218-L238

```
bool CheckProofOfWork(uint256 hash, uint32_t nBits, const Config &config) {
    bool fNegative;
    bool fOverflow;
```

```
    arith_uint256 bnTarget;

    bnTarget.SetCompact(nBits, &fNegative, &fOverflow);

    // Check range
    if (fNegative || bnTarget == 0 || fOverflow ||
        bnTarget >
            UintToArith256(config.GetChainParams().GetConsensus().powLimit)) {
        return false;
    }

    // Check proof of work matches claimed amount
    if (UintToArith256(hash) > bnTarget) {
        return false;
    }

    return true;
}
```

- src/validation.cpp#L144-L152

```
struct CBlockIndexWorkComparator {
    bool operator()(const CBlockIndex *pa, const CBlockIndex *pb) const {
        // First sort by most total work, ...
        if (pa->nChainWork > pb->nChainWork) {
            return false;
        }
        if (pa->nChainWork < pb->nChainWork) {
            return true;
        }
```

**Solution**

Evaluate migrating from pure PoW to a PoS, BFT, or hybrid consensus mechanism with economic penalties and deterministic finality so that attack cost no longer depends solely on temporarily transferable SHA-256 hashrate.

**Status**

Acknowledged; The project team stated that miner KYC, time-limited authorization, and checkpoints significantly

reduce external hashrate attacks and chain reorganization risks. The remaining risks involve malicious authorized miners, private key leakage, and hashrate concentration, requiring ongoing governance and monitoring.

### [N8] [Low] Extreme Fee-Rate Configuration May Cause a Non-Monotonic Fee Curve and Integer Overflow

**Category: Business logic security audit**

**Content**

`-mempoolminfeerate` is checked only for non-negativity and is not restricted to a value at or below the fee curve's maximum. If the configured minimum fee rate exceeds `MAX_MEMPOOL_RAMP_FEE_RATE`, the calculated value is clamped to the lower maximum after mempool usage enters the ramp region, causing the admission fee rate to decrease as usage increases. In addition, `CFeeRate::GetFee()` uses `int64_t` to calculate transaction size multiplied by fee rate. Combining an extreme fee rate with a large transaction may cause signed integer overflow, making the required fee negative or otherwise invalid. Because the fee check only handles required fees greater than zero, an invalid result may bypass the minimum fee check or cause a process failure. Default configurations do not trigger this issue; exploitation requires an extreme custom node configuration.

- src/config.cpp#L1041-L1050

```
bool GlobalConfig::SetMempoolMinFeePerKB(int64_t feePerKB, std::string* err) {
    if (LessThanZero(feePerKB, err, "Policy value for mempool minimum feerate must
not be less than 0."))
    {
        return false;
    }

    mMempoolMinFeePerKB = CFeeRate(Amount(feePerKB));

    return true;
}
```

- src/txmempool.cpp#L1936-L1965

```
CFeeRate CTxMemPool::GetMinFee(size_t sizelimit) const {
    std::shared_lock lock(smtx);
    const Config& config = GlobalConfig::GetConfig();
    const int64_t floorRate =
        config.GetMempoolMinFeePerKB().GetFeePerK().GetSatoshis();
    const size_t N1 = config.GetMempoolFeeRampStart();
    const size_t N2 = sizelimit; // hard size cap (no-trim)
    const size_t usage = DynamicMemoryUsageNL();

    // Below the ramp start (or a degenerate N1 >= N2 config) the admission
    // floor is just the configured minimum feerate.
    if (usage <= N1 || N2 <= N1) {
        return CFeeRate(Amount(floorRate));
    }
    // At/above the hard cap the pool is full. Admission is hard-rejected
    // separately before this point, so this is only a safety clamp.
    if (usage >= N2) {
        return CFeeRate(MAX_MEMPOOL_RAMP_FEE_RATE);
    }
    // Hyperbolic ramp between N1 and N2 with a pole at N2 (alpha = 1):
    //   rate = floor * (N2 - N1) / (N2 - usage)
    // As usage approaches N2 the required feerate diverges, so in practice the
    // mempool asymptotes below N2 instead of ever reaching it.
    __int128 rate = (static_cast<__int128>(floorRate) *
                     static_cast<int64_t>(N2 - N1)) /
                    static_cast<int64_t>(N2 - usage);
    if (rate > MAX_MEMPOOL_RAMP_FEE_RATE.GetSatoshis()) {
        rate = MAX_MEMPOOL_RAMP_FEE_RATE.GetSatoshis();
    }
    return CFeeRate(Amount(static_cast<int64_t>(rate)));
}
```

- src/amount.cpp#L28-L43

```
Amount CFeeRate::GetFee(size_t nBytes_) const {
    assert(nBytes_ <= uint64_t(std::numeric_limits<int64_t>::max()));
    int64_t nSize = int64_t(nBytes_);

    if (nSize < 1000) {
        nSize = 1000;
    }
```

```
    Amount nFee = nSize * nSatoshisPerK / 1000;

    if (nFee == Amount(0) && nSize != 0) {
        if (nSatoshisPerK > Amount(0)) {
            nFee = Amount(1);
        }
        if (nSatoshisPerK < Amount(0)) {
            nFee = Amount(-1);
        }
```

- src/validation.cpp#L1365-L1373

```
static bool CheckMempoolMinFee(
    const Amount& nModifiedFees,
    const Amount& nMempoolRejectFee) {
    // Check mempool minimal fee requirement.
    if (nMempoolRejectFee > Amount(0) &&
        nModifiedFees < nMempoolRejectFee) {
        return false;
    }
    return true;
}
```

**Solution**

Restrict `mempoolminfeerate` so that it cannot exceed the fee curve's maximum rate. Calculate transaction fees using `__int128`, and perform range checking or saturation before converting the result back to `int64_t`.

**Status**

Fixed

### [N9] [Low] Stale `ancestorsHeight` After Reorganization Invalidates Unconfirmed Transaction Chain Limits

**Category: Business logic security audit**

**Content**

When a chain reorganization returns a previously confirmed parent transaction to the mempool,

`UpdateTransactionsFromBlock()` restores parent-child links and reassigns `insertionIndex`, but does not recalculate `ancestorsHeight` for the affected transactions and their descendants. As a result, descendants retain cached heights below their actual values, allowing subsequent transactions to potentially bypass the `-limitancestorcount` limit, while wallets may continue selecting unconfirmed outputs that have actually exceeded the chain-depth limit. When `-checkmempool` is enabled, the incorrect cached height may also trigger consistency assertions. This risk first requires a natural reorganization, a valid higher-work chain switch, or an administrator-triggered reorganization; it cannot be triggered directly by an ordinary P2P transaction.

- src/txmempool.cpp#L163-L180, L1728-L1767

```
CEnsureNonNullChangeSet nonNullChangeSet(*this, changeSet);

// Now we will check transactions connected with newly added transactions
// We are doing this because it could be that a newly added transaction
// did not end up in the journal but have descendants which are in the journal
already.
setEntries affected = getConnectedNL(addedTransactions);

// A reorg re-adds previously-confirmed transactions, and can re-add a parent
// after a child that is already in the mempool. That leaves insertionIndex
// non-topological (the parent gets a larger index than its child). Restore a
// topological insertionIndex over the whole affected connected component
// before anything relies on the ordering it provides (journal acceptance
// below, block-template assembly, ancestorsHeight maintenance).
setEntries component { affected };
component.insert(addedTransactions.begin(), addedTransactions.end());
reassignInsertionIndicesNL(component);

checkJournalAcceptanceNL(affected, nonNullChangeSet.Get());
```

```
void CTxMemPool::UpdateAncestorsHeightNL(CTxMemPool::setEntriesTopoSorted entries)
{
    while(!entries.empty())
    {
        txiter entry = *entries.begin();
        entries.erase(entries.begin());
```

```
        // Collect children before checking - they may still need updating even if
parent is gone
        setEntries childrenToProcess;
        if(mapLinks.find(entry) != mapLinks.end())
        {
            childrenToProcess = GetMemPoolChildrenNL(entry);
        }

        // Skip further processing if entry has been removed from mempool
        if(mapLinks.find(entry) == mapLinks.end())
        {
            // Still need to process children whose parent was removed
            for(auto child: childrenToProcess)
            {
                entries.insert(child);
            }
            continue;
        }

        for(auto child: childrenToProcess)
        {
            entries.insert(child);
        }

        size_t ancestorsHeight = 0;
        for(auto parent: GetMemPoolParentsNL(entry))
        {
            ancestorsHeight = std::max(ancestorsHeight, parent->GetAncestorsHeight()
+ 1);
        }
        mapTx.modify(entry, [ancestorsHeight](CTxMemPoolEntry& entry) {
                                entry.SetAncestorsHeight(ancestorsHeight);
                            });
    }
}
```

**Solution**

After `reassignInsertionIndicesNL(component)` completes, call `UpdateAncestorsHeightNL()` for the entire affected connected component in the updated parent-before-child topological order, and then perform the journal check.

**Status**

Fixed

### [N10] [High] Fixed-Offset Miner Bill Payment Address Parsing Allows Address-Binding Bypass

**Category: Business logic security audit**

**Content**

When validating the "fixed change address," both `FilledMinerBillV2()` and the legacy `FilledMinerBill()` read 20 bytes from `tx.vout[0].scriptPubKey` at the hardcoded offset `3`, treat them as the payment address hash, and include them in the Manager-signed message, but neither function verifies that the output matches the complete P2PKH script template. The consensus-layer `CheckTransactionCommon()` and `CheckTxOutputs()` functions do not constrain the script format, while `IsStandard()` is only a mempool policy check. The bill validation also checks only the lower-bound condition `chargeValue >= chargeValueLimit` and does not verify the actual destination of the funds. Because a top-level `OP_RETURN` terminates successfully after Genesis, a miner holding a valid Manager signature can construct a non-standard script that retains the Manager-approved 20-byte hash at offset `3` while replacing the prefix with `61 61 14 <H> 6a …`. The miner can later reclaim the matured output using an empty `scriptSig`. Therefore, an authorized miner can bypass the fixed-change-address restriction, retain the KYC fee that should have been paid, and undermine the integrity of billing and compliance fund flows.

- src/validation.cpp#L810-L840, L888-L911, L1020-L1027

```
// check kyc charge rate
uint64_t totalOutputValue = tx.GetValueOut().GetSatoshis();
if (totalOutputValue == 0) {
    LogPrintf("Total output value is zero, cannot calculate charge rate\n");
    return false;
}
uint64_t chargeValueLimit = totalOutputValue * kycChargeRate / 100;
uint64_t chargeValue = tx.vout[0].nValue.GetSatoshis();
if (chargeValue < chargeValueLimit) {
    LogPrintf("Charge value is less than charge value limit !!! (%d < %d) \n",
chargeValue, chargeValueLimit);
    return false;
```

```
}

// Compute message hash
msgMiner.insert(msgMiner.end(), pubkeyMinerVec.begin(), pubkeyMinerVec.end());
msgMiner.insert(msgMiner.end(), currentChainHeightVec.begin(),
currentChainHeightVec.end());
// Use uint256.GetHex() to get the big-endian block hash
std::vector<uint8_t> tipBlockHashVec = ParseHex(tipBlockHash.GetHex());
msgMiner.insert(msgMiner.end(), tipBlockHashVec.begin(), tipBlockHashVec.end());
msgHashMiner = Hash(msgMiner.begin(), msgMiner.end());

msgManager.insert(msgManager.end(), pubkeyMinerVec.begin(), pubkeyMinerVec.end());
msgManager.insert(msgManager.end(), kycPermissionHeightVec.begin(),
kycPermissionHeightVec.end());
msgManager.insert(msgManager.end(), kycChargeRateVec.begin(),
kycChargeRateVec.end());
msgManager.insert(msgManager.end(), countryCodeVec.begin(), countryCodeVec.end());
if (isFixedChangeAddress == true) {
    uint64_t chargeAddressIndex = 3;
    if (!safeReadScript(chargeOutputScript, chargeAddressIndex, 20,
chargeAddressPubkeyHashVec)) { return false; }
    msgManager.insert(msgManager.end(), chargeAddressPubkeyHashVec.begin(),
chargeAddressPubkeyHashVec.end());
}
msgHashManager = Hash(msgManager.begin(), msgManager.end());
```

```
// get script data. skip 26 bytes of P2PKH+op_return.
outputScriptIndex += 26;

// get KYC flag
std::vector<uint8_t> kycFlagVec;
if (!safeReadScript(chargeOutputScript, outputScriptIndex, 3, kycFlagVec)) {
    return false;
}
bool ifCheckChargeAddress = false;
if (kycFlagVec[0] == 0x4C && kycFlagVec[2] == 0x01) {
    ifCheckChargeAddress = true;
}

vector<uint8_t> currentBlockHeightVec(
    coinbaseHeight.encodedHeight.begin(),
    coinbaseHeight.encodedHeight.begin() +
        coinbaseHeight.encodedHeightSize);
```

```
// get charge address script
std::vector<uint8_t> chargeAddressScript;
uint64_t chargeAddressIndex = 3;
if (!safeReadScript(chargeOutputScript, chargeAddressIndex, 20, chargeAddressScript))
{
        return false;
}

// if check charge address
if (ifCheckChargeAddress == true) {
    for(auto a : chargeAddressScript) {
        TMsgA.push_back(a);
    }
}

uint256 TMsgAHash = Hash(TMsgA.begin(), TMsgA.end());
```

**Solution**

Before parsing `chargeAddressPubkeyHash`, explicitly validate the complete P2PKH template of `scriptPubKey`: bytes `[0..2]` must be `76 a9 14`, and bytes `[23..24]` must be `88 ac`.

**Status**

Fixed

### [N11] [Information] Test-Only Block Validation Pause RPC Exposed in Production

**Category: Business logic security audit**

**Content**

The test-only `waitaftervalidatingblock` RPC is registered unconditionally in the production command table. The `hidden` category only hides help information and does not restrict invocation. A user with full RPC access who knows the hash of a target valid block in advance can add it to the waiting list, causing the corresponding validation task to pause and affecting the local node's chain-tip update and parallel block selection.

- src/rpc/blockchain.cpp#L2598-L2600, L2690-L2690

```
uint256 blockHash(uint256S(strHash));

blockValidationStatus.waitAfterValidation(blockHash, strAction);

{ "hidden",             "waitaftervalidatingblock",         waitaftervalidatingblock,
true,  {"blockhash","action"} },
```

**Solution**

Register this RPC only on regtest or in test builds. On mainnet and testnet, disable the RPC or return a method-not-found error. Do not rely on the `hidden` category as an access-control mechanism.

**Status**

Fixed

# 5 Audit Result

| Audit Number | Audit Team | Audit Date | Audit Result |
| --- | --- | --- | --- |
| 0X002608120001 | SlowMist Security Team | 2026.07.30 - 2026.08.12 | Low Risk |

Summary conclusion: The SlowMist security team use a manual and SlowMist team's analysis tool to audit the project, during the audit work we found 1 high risk, 5 low risk vulnerabilities, 1 suggestion, and 4 informational findings. The high-risk vulnerability, 4 low risk vulnerabilities, the suggestion, and 1 informational finding have been fixed. All remaining findings have been acknowledged by the project team.

# 6 Statement

SlowMist issues this report with reference to the facts that have occurred or existed before the issuance of this report, and only assumes corresponding responsibility based on these.

For the facts that occurred or existed after the issuance, SlowMist is not able to judge the security status of this project, and is not responsible for them. The security audit analysis and other contents of this report are based on the documents and materials provided to SlowMist by the information provider till the date of the insurance report (referred to as "provided information"). SlowMist assumes: The information provided is not missing, tampered with, deleted or concealed. If the information provided is missing, tampered with, deleted, concealed, or inconsistent with the actual situation, the SlowMist shall not be liable for any loss or adverse effect resulting therefrom. SlowMist only conducts the agreed security audit on the security situation of the project and issues this report. SlowMist is not responsible for the background and other conditions of the project.

**Official Website**

www.slowmist.com

**E-mail**

team@slowmist.com

**Twitter**

@SlowMist_Team

**Github**

https://github.com/slowmist